Relazione di progetto

Monopoly: Programmazione ad Oggetti

Matteo Manganiello

Settembre 2026

Indice

1 Analisi **2**
1.1 Descrizione e requisiti 2
1.2 Modello del Dominio 5

2 Design **7**
2.1 Architettura 7
2.2 Design dettagliato 8
2.2.1 Le caselle e il polimorfismo 8
2.2.2 La comunicazione con le view: il pattern Observer . . . 10
2.2.3 La cronaca testuale: il Template Method 10
2.2.4 Lo svolgimento del turno: una piccola macchina a stati 11
2.2.5 I collaboratori delle caselle senza variabili globali . . . 11
2.2.6 Il salvataggio della partita 12

3 Sviluppo **16**
3.1 Testing automatizzato 16
3.2 Note di sviluppo 17

4 Commenti finali **18**
4.1 Autovalutazione e lavori futuri 18
4.2 Difficoltà incontrate 19

Capitolo 1

Analisi

In questo capitolo sono descritti i requisiti del sistema, ossia ciò che il programma deve essere in grado di fare, e il modello del dominio, ovvero gli elementi di cui il gioco si occupa e le relazioni che li legano. Non vengono ancora affrontati aspetti implementativi, trattati nel capitolo di design.

1.1 Descrizione e requisiti

Il progetto consiste nella realizzazione di una versione digitale del gioco da tavolo Monopoly, fruibile come applicazione desktop. La modalità di gioco è locale: tutti i partecipanti utilizzano lo stesso computer, alternandosi al turno (la modalità comunemente indicata come *hot-seat*), senza necessità di connessione di rete né presenza di avversari controllati dal sistema. I giocatori si muovono a turno su un tabellone di quaranta caselle in base al lancio di due dadi, acquistano le proprietà su cui si fermano, riscuotono gli affitti dagli altri partecipanti, versano le tasse previste e possono essere inviati in prigione; un giocatore che non dispone più di liquidità sufficiente fallisce ed è escluso dalla partita. La partita termina quando rimane un solo giocatore non fallito, che risulta vincitore.

Di seguito sono richiamati alcuni termini utilizzati nel seguito della relazione:

- **casella**: una delle quaranta posizioni del tabellone; ciascuna casella appartiene a una tipologia (proprietà, tassa, prigione, "Via", ecc.) e produce un effetto diverso quando un giocatore vi si ferma.
- **proprietà**: una casella acquistabile. Ne esistono di tre tipologie: i *terreni* (caratterizzati da un colore), le *stazioni* e le *società*.

- **monopolio di colore**: situazione in cui un giocatore possiede tutti i terreni dello stesso colore; in tal caso l'affitto dovuto su quei terreni risulta raddoppiato.
- **doppio**: lancio in cui i due dadi restituiscono lo stesso valore. Il giocatore che ottiene un doppio effettua un ulteriore lancio.

Requisiti funzionali

Di seguito sono elencati i requisiti funzionali del sistema.

- Il sistema deve consentire la preparazione di una nuova partita, con la scelta dei giocatori e della rispettiva pedina, assegnando a ciascuno un capitale iniziale di 1500.
- Il sistema deve gestire lo svolgimento dei turni: al giocatore di turno spetta il lancio dei dadi, lo spostamento sul tabellone (circolare: superata l'ultima casella si ritorna alla prima) e la risoluzione dell'effetto della casella di arrivo.
- Il sistema deve applicare la regola dei doppi: chi ottiene un doppio effettua un ulteriore lancio; al terzo doppio consecutivo il giocatore viene inviato in prigione.
- Il sistema deve consentire l'acquisto delle proprietà: quando un giocatore si ferma su una proprietà libera che può permettersi, gli viene proposto l'acquisto, la cui decisione spetta al giocatore.
- Il sistema deve calcolare e riscuotere gli affitti in base alla tipologia di proprietà: sui terreni l'affitto raddoppia in presenza del monopolio di colore, sulle stazioni assume i valori 25, 50, 100 o 200 in base al numero di stazioni possedute dallo stesso proprietario, sulle società dipende dal valore dei dadi appena lanciati (moltiplicatore 4 con una sola società posseduta, 10 con entrambe).
- Il sistema deve far versare le tasse previste e accreditare lo stipendio (200) al giocatore che transita o si ferma sulla casella "Via".
- Il sistema deve gestire la prigione: l'ingresso avviene con tre doppi consecutivi o tramite la casella "Vai in prigione"; l'uscita avviene ottenendo un doppio oppure pagando una cauzione di 50, che diventa obbligatoria dopo tre tentativi falliti.
- Il sistema deve gestire il fallimento: un giocatore che non è in grado di saldare un debito cede l'intero patrimonio (liquidità e proprietà) al creditore ed è escluso dalla partita; le proprietà eventualmente tornate alla banca risultano nuovamente acquistabili.
- Il sistema deve dichiarare conclusa la partita quando rimane un solo giocatore non fallito.

- Il sistema deve consentire il salvataggio della partita su file e il relativo caricamento in un momento successivo.

Requisiti non funzionali

Di seguito sono elencati i requisiti non funzionali del sistema.

- Il sistema deve funzionare su un singolo computer, senza connessione di rete e senza richiedere l'installazione di un server o di un database.
- L'interfaccia deve mostrare in ogni momento, in modo chiaro, la situazione economica, le proprietà possedute, la posizione sul tabellone e il turno di ciascun giocatore.
- L'interfaccia deve abilitare i comandi soltanto quando l'azione corrispondente è effettivamente consentita (ad esempio, il lancio dei dadi deve risultare disponibile solo quando spetta al giocatore di turno), evitando che l'utente possa selezionare un'operazione non valida in quel momento.
- L'uso dell'applicazione non deve richiedere la lettura di un manuale: comandi, etichette e messaggi devono essere autoesplicativi anche per un utente che utilizza il programma per la prima volta.
- Il sistema deve fornire un riscontro immediato per ogni azione compiuta dal giocatore (lancio dei dadi, acquisto, pagamento di un affitto), in modo che l'esito sia sempre chiaro prima di procedere con l'azione successiva.
- In caso di errore, ad esempio un file di salvataggio danneggiato o incompleto, il sistema deve segnalarlo con un messaggio comprensibile, senza terminare in modo anomalo.
- Il sistema deve garantire l'integrità dei dati salvati: un file di salvataggio corrotto o relativo a una versione differente deve essere riconosciuto come tale, senza compromettere la partita in corso né restituire uno stato di gioco incoerente.
- L'architettura deve favorire l'estensibilità: l'introduzione di un nuovo tipo di casella o di una nuova regola non deve richiedere la modifica delle parti già esistenti.
- La logica di gioco deve poter essere disaccoppiata dall'interfaccia (ad esempio, sostituendo l'interfaccia grafica con una a riga di comando) senza necessità di essere riscritta.
- I test automatizzati devono produrre risultati deterministici; la componente casuale dei dadi deve pertanto poter essere resa prevedibile in fase di test.

1.2 Modello del Dominio

L'elemento centrale del dominio applicativo è la *partita*, che costituisce il contesto complessivo in cui interagiscono gli altri elementi. La partita si svolge su un *tabellone*, costituito da un elenco ordinato di *caselle*: alcune sono *proprietà* acquistabili (terreni, stazioni e società), altre impongono il pagamento di una *tassa*, altre ancora costituiscono la *prigione* o determinano l'ingresso in prigione; vi è inoltre la casella "Via", che eroga lo stipendio, e alcune caselle di semplice transito.

Alla partita partecipano i *giocatori*, ciascuno caratterizzato da un nome, una *pedina*, una quantità di denaro, una posizione sul tabellone, l'insieme delle proprietà possedute e uno stato (in gioco, in prigione oppure fallito). La gestione del denaro è affidata alla *banca*, unico soggetto autorizzato a far entrare o uscire denaro dal patrimonio dei giocatori e detentrice iniziale di tutte le proprietà. I *dadi* stabiliscono l'entità dello spostamento e, in caso di doppio, influenzano lo svolgimento del turno.

Le *regole* del gioco non costituiscono un'entità autonoma, bensì i vincoli che disciplinano i rapporti tra le entità sopra descritte; per tale motivo sono trattate nel capitolo di design. La Figura 1.1 riassume le entità principali del dominio e le relazioni intercorrenti tra loro.

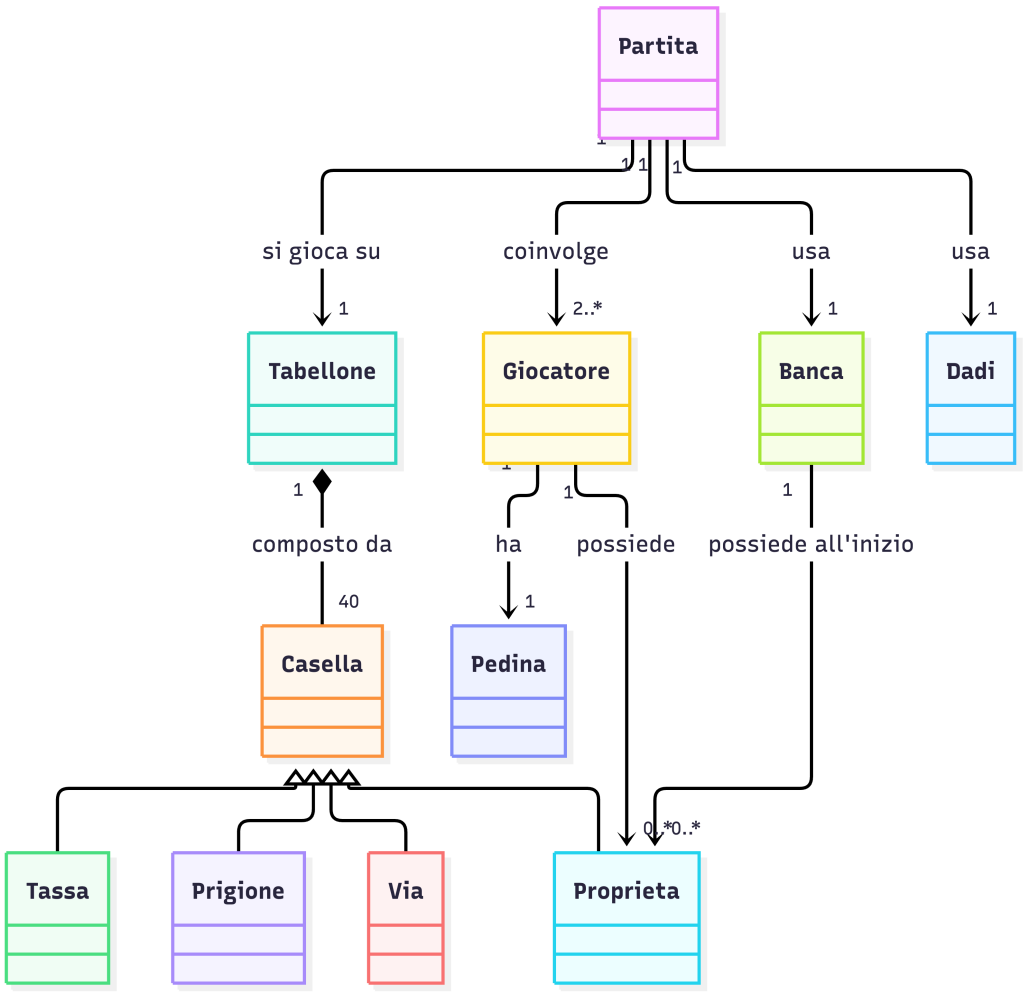


Figura 1.1: Schema del modello del dominio: le entità principali del gioco e i loro rapporti.

Capitolo 2

Design

In questo capitolo è illustrata l'organizzazione del codice adottata per soddisfare i requisiti descritti in precedenza: dapprima la struttura architetturale generale, quindi l'approfondimento di alcuni aspetti di design ritenuti particolarmente rilevanti.

2.1 Architettura

L'architettura del sistema segue il modello **MVC** (Model-View-Controller), articolato in tre componenti principali.

- Il **model** (package `it.unibo.monopoly.model`) contiene le regole e i dati del gioco ed è organizzato in sottopacchetti tematici: `board` per il tabellone e le caselle, `economy` per la banca e le proprietà, `game` per i turni e i dadi, `player` per i giocatori e `persistence` per il salvataggio. Il model non dipende da alcuna componente grafica.
- La **view** (package `view`) si occupa della rappresentazione grafica della partita e della raccolta degli input dell'utente. Non contiene alcuna regola di gioco: le azioni dell'utente vengono inoltrate al controller, e la view attende di essere notificata delle variazioni di stato da rappresentare.
- Il **controller** (package `controller`) è costituito dalla classe `GameEngine`, punto di accesso unico per il comando della partita (`startGame`, `rollDice`, `endTurn`, ...): riceve i comandi, ne delega l'esecuzione al model e successivamente notifica le view.

La view e il model non comunicano mai direttamente tra loro, ma esclusivamente per il tramite del controller e di un'interfaccia neutra, `GameObserver`: non conoscendo alcuna classe della componente grafica, il model consente la sostituzione dell'interfaccia senza doverlo modificare. Si

tratta di una proprietà verificata concretamente nel progetto, nel quale due view distinte, la finestra grafica e una cronaca testuale sul terminale, ricevono gli stessi eventi senza che il motore di gioco debba conoscerne l'identità. La Figura 2.1 illustra lo schema architetturale.

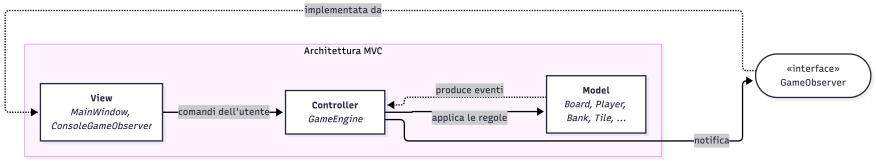


Figura 2.1: Schema architetturale MVC. Il `GameEngine` è il controller; le view si registrano come `GameObserver` e ricevono gli aggiornamenti senza che il motore conosca le loro classi.

2.2 Design dettagliato

In questa sezione sono approfondite singolarmente alcune scelte di design. Per ciascuna vengono descritti il problema affrontato, la soluzione adottata (con le principali alternative considerate e scartate) e, ove opportuno, uno schema esplicativo e il pattern di progettazione utilizzato.

2.2.1 Le caselle e il polimorfismo

Problema. Le caselle del tabellone presentano comportamenti eterogenei: una proprietà determina l'acquisto o il pagamento di un affitto, una casella tassa comporta un versamento, un'altra determina l'ingresso in prigione, altre ancora non producono alcun effetto. Il componente responsabile dello spostamento delle pedine non dovrebbe tuttavia dover conoscere, di volta in volta, la tipologia specifica della casella.

Soluzione. È stata definita una classe astratta `Tile` dotata di un metodo astratto `onLand(Player)`, e una sottoclasse concreta per ciascuna tipologia di casella (`PropertyTile`, `TaxTile`, `JailTile`, `GoToJailTile`, `StartTile`, `FreeParkingTile`), ognuna delle quali implementa in `onLand` esclusivamente il proprio effetto specifico. Il gestore dei turni invoca pertanto sempre e soltanto `tile.onLand(player)`, senza necessità di conoscere l'esito dell'operazione. L'alternativa scartata, un'unica classe `Tile` dotata di un campo "tipo" e di una struttura condizionale (`if`/`switch`) in `onLand`, avrebbe richiesto di intervenire su quel metodo a ogni nuova tipologia di casella introdotta, vanificando proprio il vantaggio offerto dal polimorfismo.

Le proprietà presentano un ulteriore livello di specializzazione, poiché anche il calcolo dell'affitto varia in base alla tipologia: sui terreni l'importo raddoppia in presenza del monopolio di colore, sulle stazioni dipende dal numero di stazioni possedute dallo stesso proprietario, sulle società dipende dal valore dei dadi. Per tale ragione `PropertyTile` espone un metodo `computeRent`, ridefinito in modo differente da `StreetTile`, `StationTile` e `UtilityTile` (Figura 2.2). Il componente che gestisce il pagamento dell'affitto invoca `computeRent` senza necessità di conoscere quale delle tre implementazioni verrà effettivamente eseguita. Le caselle "Imprevisti" e "Probabilità" sono realizzate da `CardTile`, ulteriore sottoclasse diretta di `Tile`, il cui metodo `onLand` fa pescare al giocatore una carta che comporta un incasso o un pagamento.

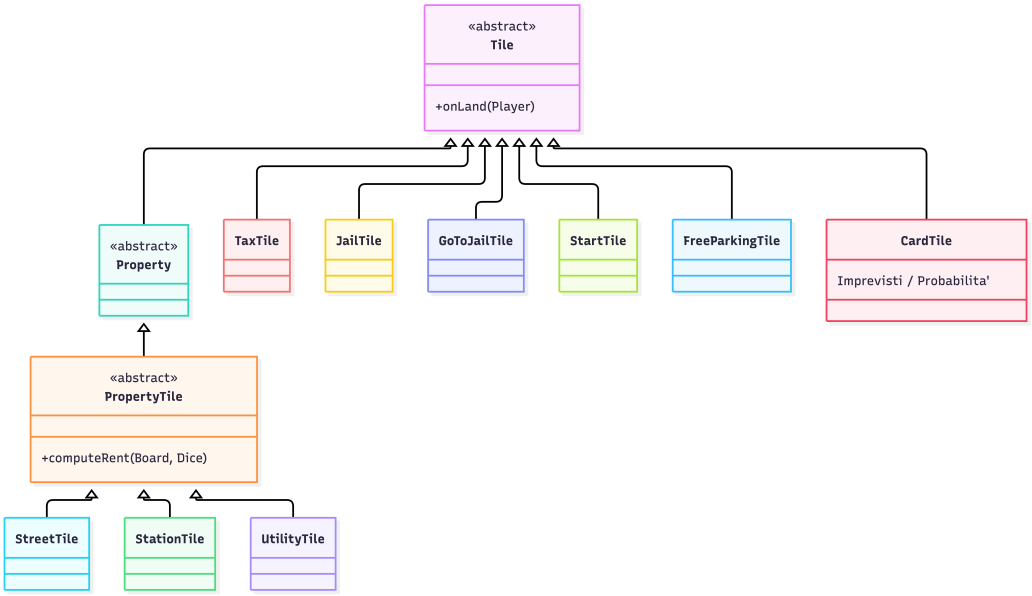


Figura 2.2: Gerarchia delle caselle. `onLand` è ridefinito da ogni tipo; per le proprietà anche `computeRent` cambia a seconda che sia un terreno, una stazione o una società.

Separazione tra dato e comportamento. La classe `Property`, appartenente al dominio, descrive *che cosa* sia una proprietà (nome, prezzo, affitto, proprietario) e rimane un oggetto passivo; è la classe `PropertyTile` ad aggiungere *come* essa si comporti durante la partita, funzionalità che richiede l'accesso alla banca. Tale separazione è stata mantenuta deliberatamente, così che la parte di dominio resti semplice e possa essere sottoposta a test in isolamento, senza coinvolgere la banca.

2.2.2 La comunicazione con le view: il pattern Observer

Problema. Ogni evento significativo della partita (un lancio di dadi, un acquisto, il pagamento di un affitto) richiede l'aggiornamento delle view. Il model e il controller non devono tuttavia dipendere dalle classi della componente grafica, pena l'impossibilità di sostituire l'interfaccia; inoltre le view in ascolto possono essere più di una contemporaneamente.

Soluzione. È stato adottato il *pattern Observer*: il `GameEngine` riveste il ruolo di soggetto osservato, e ogni componente interessata a essere notificata implementa l'interfaccia `GameObserver` registrandosi tramite `addObserver`. Il motore invoca i metodi dell'interfaccia senza conoscere l'identità dell'osservatore (Figura 2.3). Le alternative scartate, l'invocazione diretta della finestra da parte del model oppure un controllo periodico, da parte della view, dello stato del model, avrebbero introdotto rispettivamente una dipendenza dalla grafica e un meccanismo dispendioso e meno chiaro.

Tutti i metodi di `GameObserver` presentano un'implementazione di default vuota, in modo che ciascuna view ridefinisca soltanto gli eventi di proprio interesse. Un aspetto rilevante riguarda l'ordine di consegna dei messaggi: gli effetti delle caselle si verificano *durante* il lancio dei dadi, per cui una notifica immediata li mostrerebbe prima ancora del messaggio relativo al lancio stesso. Per tale motivo gli eventi prodotti dal model vengono accodati e pubblicati congiuntamente al termine dell'esecuzione del comando, rispettando l'ordine cronologico atteso.

2.2.3 La cronaca testuale: il Template Method

Problema. La partita viene raccontata riga per riga da una view testuale, `ConsoleGameObserver`, che stampa sulla console messaggi quali "Alice compra Vicolo Corto per 60". La finestra grafica non necessita di una propria cronaca testuale, poiché ciò che accade è già visibile da pedine, saldi e proprietà mostrati a schermo; è tuttavia ragionevole prevedere che in futuro possa rendersi utile un'ulteriore destinazione per questi stessi messaggi (ad esempio un file di log), senza dover duplicare la logica di formattazione già scritta per la console.

Soluzione. È stato adottato il *pattern Template Method*. La classe astratta `TextGameObserver` definisce *il contenuto* da scrivere per ciascun evento, delegando a un unico metodo da implementare, `write(String)`, la responsabilità di stabilire *la destinazione* della scrittura (Figura 2.4). `ConsoleGameObserver` è oggi l'unica sottoclasse concreta e si limita a implementare `write` stampando sullo standard output; l'eventuale aggiunta di una nuova destinazione richiederebbe soltanto una sottoclasse di poche righe,

senza toccare `TextGameObserver`. L'alternativa scartata, scrivere la formattazione direttamente dentro `ConsoleGameObserver` senza classe intermedia, avrebbe legato il contenuto della cronaca a una singola destinazione, rendendo necessaria una riscrittura, e non una semplice estensione, qualora se ne aggiungesse una seconda.

2.2.4 Lo svolgimento del turno: una piccola macchina a stati

Problema. Lo svolgimento di un turno segue una sequenza predefinita: lancio dei dadi, spostamento, applicazione dell'effetto della casella di arrivo, e successivamente un nuovo lancio (in presenza di un doppio) oppure il passaggio del turno. A ciò si aggiunge un caso particolare: quando un giocatore si ferma su una proprietà libera, il sistema deve *sospendere l'esecuzione in attesa* della sua decisione di acquisto, senza tuttavia bloccare l'interfaccia grafica.

Soluzione. Il turno è stato modellato come una macchina a stati, mediante l'enumerazione `GamePhase` (`ROLL`, `MOVE`, `ACTION`, `AWAITING_PURCHASE_DECISION`, `END_TURN`) e la classe `TurnManager`, responsabile della transizione tra le fasi (Figura 2.5). L'attesa della decisione di acquisto è trattata come una fase a tutti gli effetti: anziché sospendere l'esecuzione fino alla risposta dell'utente, la partita transita nella fase di attesa e vi permane finché il giocatore non risponde, senza poter lanciare i dadi né terminare il turno. L'alternativa scartata, sospendere effettivamente l'esecuzione in attesa dell'interazione dell'utente, avrebbe bloccato l'interfaccia grafica, condizione da evitare in un'applicazione a finestre.

La rappresentazione delle fasi mediante enumerazione presenta un ulteriore vantaggio: la view è sempre in grado di determinare lo stato corrente del turno e di abilitare o disabilitare i comandi di conseguenza (il pulsante "Tira i dadi" risulta attivo esclusivamente nella fase `ROLL`).

2.2.5 I collaboratori delle caselle senza variabili globali

Problema. Il metodo `onLand` riceve come unico parametro il giocatore, ma per produrre il proprio effetto una casella necessita spesso di ulteriori collaboratori: per il pagamento di un affitto è necessaria la banca, per l'invio in prigione sono necessarie le relative regole. È necessario stabilire in che modo tali collaboratori vengano resi disponibili alla casella.

Soluzione. Tali collaboratori vengono forniti alla casella all'atto della sua costruzione, raccolti in un unico oggetto `GameContext` (banca, regole

economiche, regole della prigione, canale degli eventi). Al momento dell'atterraggio è pertanto sufficiente il riferimento al giocatore, e tutte le caselle operano sulla medesima banca. È stato deliberatamente evitato il pattern *Singleton*, che pure sarebbe stato pratico: un Singleton costituisce di fatto una variabile globale, e avrebbe impedito, ad esempio, di gestire più partite distinte nello stesso programma o di predisporre una banca fittizia nei test. Il passaggio dei collaboratori tramite costruttore richiede un maggior numero di parametri, ma mantiene le dipendenze esplicite e sotto controllo.

2.2.6 Il salvataggio della partita

Problema. È necessario poter salvare la partita su file e ricaricarla successivamente, adottando preferibilmente un formato leggibile e senza introdurre librerie esterne; il caricamento non deve inoltre generare comportamenti anomali qualora il file risulti danneggiato o appartenente a una versione differente.

Soluzione. Vengono salvati esclusivamente i dati soggetti a variazione durante la partita (i giocatori, il turno corrente, la fase, la cassa della banca): il tabellone, essendo sempre identico, viene ricostruito in fase di caricamento, e delle caselle è sufficiente conoscere l'eventuale proprietario. Tale "fotografia" è rappresentata da un `record` immutabile, `SaveData`, scritto su file nel formato `.properties`, un formato a coppie chiave-valore già supportato nativamente da Java, che non richiede librerie aggiuntive ed è apribile con un comune editor di testo. In testa al file sono presenti un identificativo e un numero di versione, così che eventuali salvataggi di formato differente risultino riconosciuti come incompatibili anziché letti in modo errato, e ogni dato mancante o non valido viene segnalato con un messaggio esplicativo anziché causare l'arresto anomalo del programma. Sono state scartate sia la serializzazione automatica di Java (file poco leggibili e fragili rispetto a modifiche delle classi), sia l'adozione del formato JSON tramite libreria esterna (dipendenza aggiuntiva non necessaria in questo contesto).

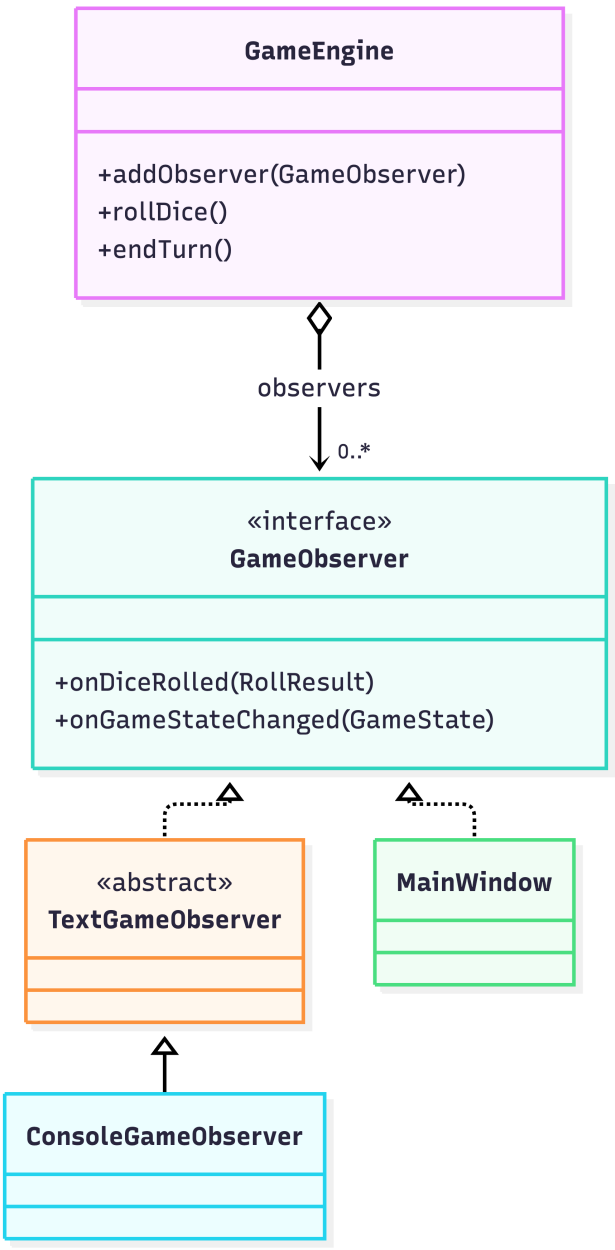


Figura 2.3: Il pattern Observer: il `GameEngine` avvisa gli osservatori registrati (la finestra grafica e la cronaca testuale) tramite l'interfaccia `GameObserver`.

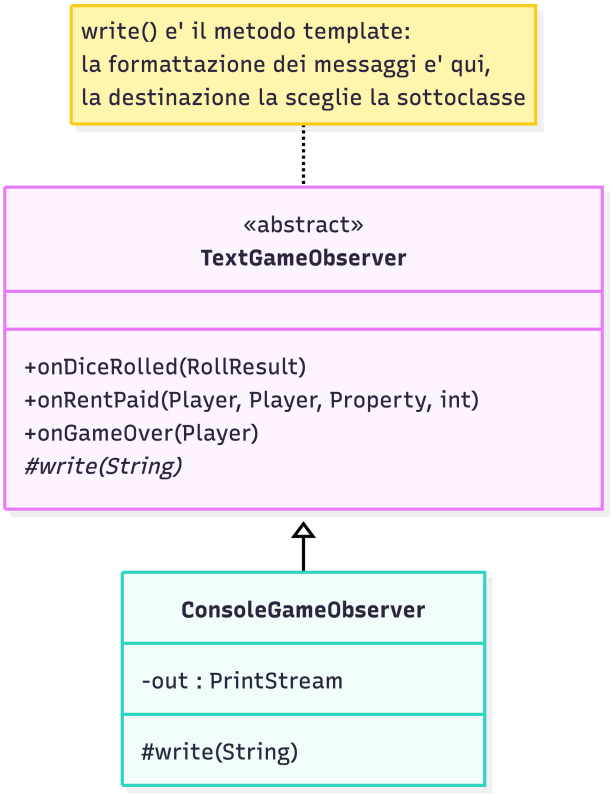


Figura 2.4: Template Method: `TextGameObserver` definisce il contenuto dei messaggi, mentre la sottoclasse `ConsoleGameObserver` decide solo dove scriverli, implementando il metodo template `write`.

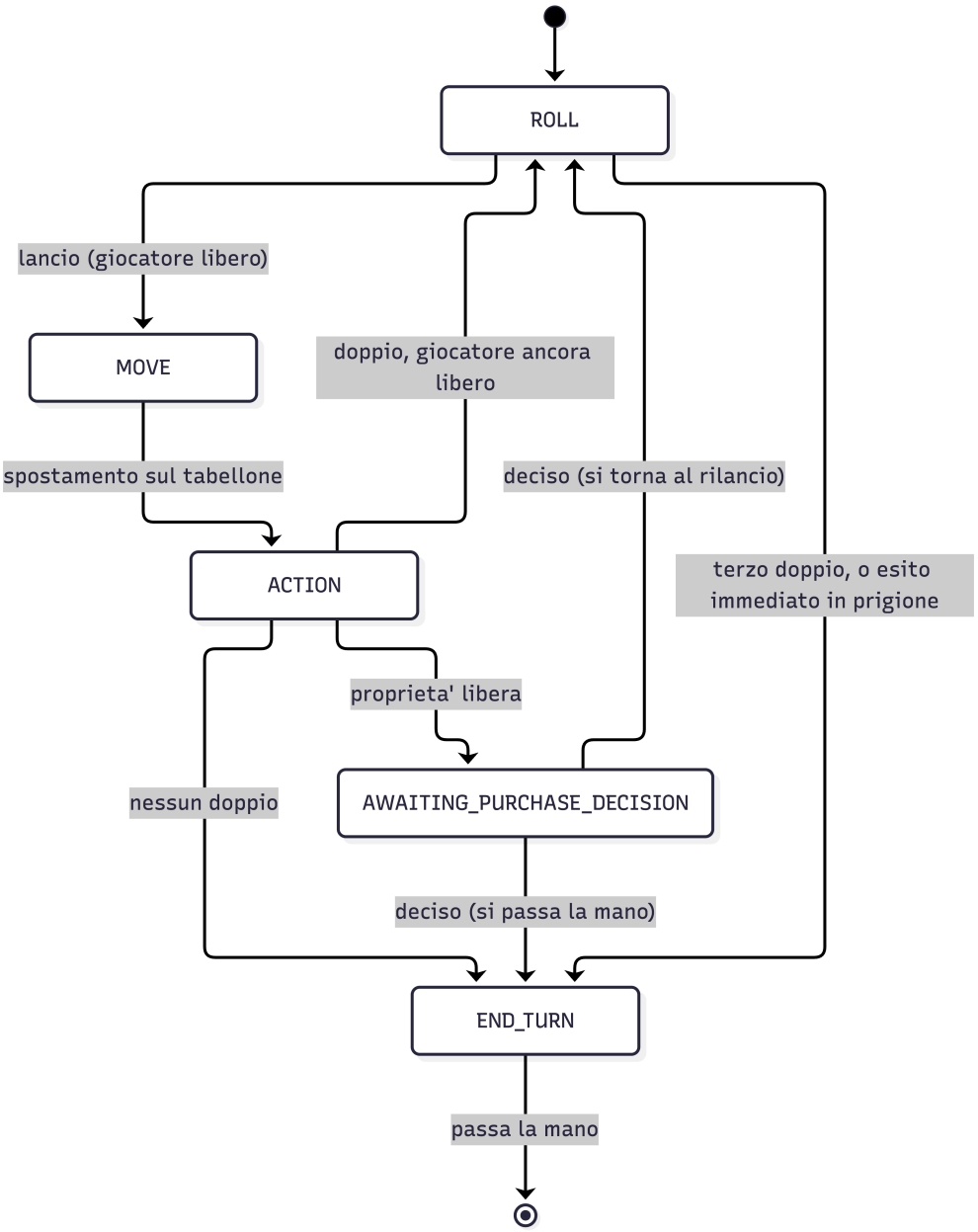


Figura 2.5: Le fasi di un turno (`GamePhase`) e i passaggi fra loro. La fase di attesa dell'acquisto permette di fermare il turno senza bloccare il programma.

Capitolo 3

Sviluppo

3.1 Testing automatizzato

I test sono stati realizzati con **JUnit 5**, per un totale di oltre 150 casi di test, eseguibili tramite `./gradlew test`. Per garantirne la ripetibilità, nei test in cui interviene la componente aleatoria dei dadi è stato utilizzato un generatore casuale a seme fisso, così che ogni lancio "casuale" produca sempre lo stesso esito e il test risulti deterministico.

I test riguardano principalmente la logica di gioco e sono organizzati per area tematica:

- **caselle ed effetti**: verifica dell'acquisto di una proprietà libera, del pagamento dell'affitto tra due giocatori, delle tasse, dello stipendio erogato dalla casella "Via" e della casella "Vai in prigione" (`TileEffectsTest`, `RentRulesTest`, `BoardTest`);
- **economia**: pagamenti tra giocatori, movimenti di denaro da e verso la banca, gestione del fallimento (`BankTest`, `EconomyManagerTest`);
- **turni, dadi e prigione**: regola dei doppi, invio in prigione dopo tre doppi consecutivi, uscita tramite doppio o cauzione, passaggio al giocatore successivo con esclusione dei falliti (`DiceTest`, `TurnManagerTest`, `TurnManagerEconomyTest`, `JailManagerTest`, `GameStateTest`);
- **flusso completo del controller**: avvio della partita, acquisto interattivo, salvataggio e caricamento (`GameEngineTest`, `GameEnginePurchaseTest`, `GameEnginePersistenceTest`, `GameSetupTest`);
- **persistenza**: verifica che una partita salvata e successivamente ricaricata risulti identica a quella originale (`GameStatePersistenceTest`).

L'interfaccia grafica non è coperta da test automatici: si è scelto di concentrare il tempo disponibile sui test della logica applicativa, ambito in cui un eventuale errore comporta conseguenze più rilevanti, verificando invece la

componente grafica manualmente attraverso l'uso dell'applicazione.

3.2 Note di sviluppo

Di seguito sono elencate le funzionalità avanzate del linguaggio Java utilizzate nel progetto, con il relativo riferimento al codice sorgente. I permalink fanno riferimento a un commit specifico e andranno rigenerati qualora il codice venisse successivamente spostato.

- **Record**: usati per le "fotografie" immutabili della partita e dei lanci di dadi: `SaveData`, `RollResult`.
- **Stream e lambda**: usati per contare i terreni dello stesso colore e le stazioni possedute da un giocatore, nel calcolo dell'affitto: `StreetTile`.
- **Optional**: usato per rappresentare l'assenza di un proprietario di una proprietà, evitando il rischio di `NullPointerException`: `Property.getOwner`.
- **Generics con parametro vincolato** (`<E extends Enum<E>>`): un unico metodo generico legge qualsiasi enum dal file di salvataggio, validandone il valore: `SaveFileFormat.readEnum`.
- **Switch *expression* esaustivo su enum**: usato per descrivere l'esito di un lancio di dadi: `TextGameObserver`.
- **Interfacce funzionali e `Consumer`**: gli eventi di gioco sono rappresentati come funzioni, accodate ed eseguite in un secondo momento: `GameEventSupport`.
- **Parti della libreria JDK non trattate a lezione**: `java.util.Properties`, per il formato di salvataggio: `SaveFileFormat`; inoltre la *toolchain* di Gradle, che permette di costruire il progetto con Java 25 anche su una macchina con un JDK differente.

Capitolo 4

Commenti finali

4.1 Autovalutazione e lavori futuri

Ritengo soddisfacente la struttura complessiva del progetto. La separazione MVC si è dimostrata solida: il model non dipende dalla componente grafica, come dimostra il fatto che le medesime regole servono sia la finestra sia la cronaca testuale sul terminale. Il polimorfismo applicato alle caselle ha permesso di introdurre le diverse tipologie di proprietà (terreni, stazioni, società) senza intervenire sul motore dei turni, e i pattern adottati (Observer, Template Method) sono scaturiti da problemi concreti dello sviluppo, così come la scelta di evitare il Singleton per la banca. L'aspetto meglio presidiato resta la logica di gioco, verificata da oltre 150 test.

L'aspetto più debole è la componente grafica: il funzionamento è corretto, ma risulta statico (le pedine si spostano direttamente sulla casella di arrivo anziché percorrere visivamente il tabellone) e l'estetica complessiva è essenziale. Sono state inoltre volutamente escluse alcune caratteristiche del Monopoly originale: la costruzione di case e alberghi, le carte "Imprevisti" e "Probabilità" che spostano la pedina (quelle presenti comportano soltanto un incasso o un pagamento) e l'asta tra gli altri partecipanti quando un giocatore rifiuta una proprietà.

Un eventuale proseguimento partirebbe proprio da questi aspetti: case e alberghi, che si integrerebbero agevolmente nella gerarchia dei terreni già esistente, le carte con effetti di movimento, il meccanismo dell'asta e l'animazione delle pedine. Come sviluppo più ambizioso, la separazione tra model e view già presente nell'architettura dovrebbe rendere possibile, in futuro, svincolare la partita dal singolo computer per una fruizione in rete, senza una revisione radicale del sistema.

4.2 Difficoltà incontrate

La difficoltà più significativa è stata la gestione dell'acquisto interattivo: inizialmente sembrava risolvibile con poche righe di codice, ma è emerso che sospendere l'esecuzione del programma in attesa della risposta avrebbe bloccato l'interfaccia grafica. Trattare l'attesa come una fase del turno, anziché come un'operazione bloccante, è stato l'aspetto da cui è derivato il maggiore apprendimento.

Un ulteriore aspetto approfondito è stato l'ordine di presentazione dei messaggi: poiché gli effetti delle caselle si verificano durante il lancio dei dadi, farli comparire nell'ordine corretto ha richiesto di accodare gli eventi anziché mostrarli immediatamente. Non è stato inoltre immediato stabilire la collocazione più opportuna della logica economica; la scelta finale, separare la banca (responsabile del trasferimento del denaro) dalle regole economiche (che stabiliscono chi debba pagare chi), ha successivamente semplificato la scrittura dei test.

Nel complesso, affrontare in autonomia tutte le fasi del progetto (analisi, design, sviluppo, test) è risultato impegnativo ma formativo: la parte più onerosa non è stata la scrittura del codice, quanto la decisione sulla sua corretta collocazione all'interno dell'architettura.